

实验：使用 GCN 预测物质溶解度

1. 配置 conda 环境

- (1) 打开命令行，创建 conda 环境

```
PS D:\progress\20240901-课题组\事务\知识工程\gcn> conda create -n gcn python=3.10
```

- (2) 激活 conda 环境

```
PS D:\progress\20240901-课题组\事务\知识工程\gcn> conda activate gcn
```

- (3) 安装 dmol 依赖库：由于 dmol 依赖库中依赖链比较老，安装时不断碰到构建依赖兼容问题，所以某些依赖需要先单独安装处理。顺序执行以下命令：

- a) 降低 pip 版本，兼容旧包

```
(gcn) C:\Users\lyx>python -m pip install "pip<24.1"
```

- b) 安装依赖

```
(gcn) C:\Users\lyx>python -m pip install vcs-versioning setuptools-scm
```

- c) 安装依赖

```
(gcn) C:\Users\lyx>python -m pip install --use-pep517 sequeval==1.2.2
```

- d) 允许安装废弃的 sklearn 包

```
(gcn) C:\Users\lyx>set SKLEARN_ALLOW_DEPRECATED_SKLEARN_PACKAGE_INSTALL=True
```

- e) 安装指定版本的 dmol-book（安装时间会稍微久一点）

```
(gcn) C:\Users\lyx>python -m pip install dmol-book==1.3.1
```

- (4) 修复 dmol 库的字体加载

由于 dmol 的 `__init__.py` 会强制在 github 上下载字体文件，考虑到网络问题，我们已经提前下载好了离线文件，但是需要修改 dmol 的加载逻辑，使得其能够使用离线文件。

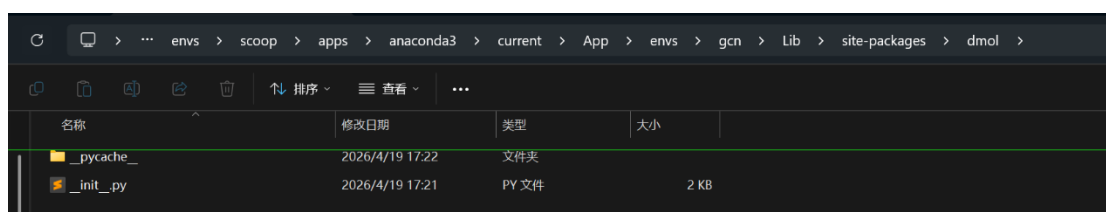
1) 定位 dmol 位置

找到自己环境所在的存储目录，记住环境所在的根目录，如本案例中即为*所标记的路径。

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> conda env list

# conda environments:
#
# * -> active
# + -> frozen
base                                D:\envs\scoop\apps\anaconda3\2025.12-2\App
base                                D:\envs\scoop\apps\anaconda3\current\App
extract                             D:\envs\scoop\apps\anaconda3\current\App\envs\extract
gcn                                  * D:\envs\scoop\apps\anaconda3\current\App\envs\gcn
kdd                                  D:\envs\scoop\apps\anaconda3\current\App\envs\kdd
multipre                             D:\envs\scoop\apps\anaconda3\current\App\envs\multipre
plt                                  D:\envs\scoop\apps\anaconda3\current\App\envs\plt
requests                             D:\envs\scoop\apps\anaconda3\current\App\envs\requests
```

打开<刚才找到的根目录>\Lib\site-packages\dmol 这个目录，找到其中 __init__.py 文件。



打开 __init__.py，并按照图示进行修改。

```
block = BlockLogs()
random.seed(0)
np.random.seed(0)
IPythonConsole.ipython_useSVG = True
IPythonConsole.drawOptions.useBWAtoPalette()
IPythonConsole.drawOptions.drawMolsSameScale = False
warnings.filterwarnings("ignore")

# urllib.request.urlretrieve(
#     "https://github.com/google/fonts/raw/main/ofl/courierprime/CourierPrime-Regular.ttf",
#     "CourierPrime-Regular.ttf",
# )

# replace with the following code
import os
if not os.path.exists("CourierPrime-Regular.ttf"):
    urllib.request.urlretrieve(
        "https://github.com/google/fonts/raw/main/ofl/courierprime/CourierPrime-Regular.ttf",
        "CourierPrime-Regular.ttf",
    )

fe = mpl.font_manager.FontEntry(fname="CourierPrime-Regular.ttf", name="courierprime")
mpl.font_manager.fontManager.ttflist.append(fe)
color_cycle = ["#444444", "#1B8C9B", "#a895bb", "#F06060", "#F3B562", "#80cedb"]
```

(5) 安装其他依赖库和修复版本依赖

lxml_html_clean 库已经从 lxml 中分离，因此需要单独安装。

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> pip install lxml_html_clean
```

setuptools 从 82.0+ 版本开始，删除了 pkg_resources 模块，因此需要重新安装 82.0 版本一下的 setuptools。

```
(gcn) C:\Users\lyx>pip install "setuptools<82.0"
```

- (6) 将 conda 配置到 notebook

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> conda install ipykernel
```

把环境作为内核注册到 notebook

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> python -m ipykernel install --user --name="gcn" --display-name="gcn"
```

- (7) 检查配置是否成功，出现 gcn 即为成功。

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> jupyter kernelspec list
Available kernels:
  python3      D:\envs\scoop\apps\anaconda3\current\AppData\Local\Envs\gcn\share\jupyter\kernels\python3
  dmo1         C:\Users\luobo\AppData\Roaming\jupyter\kernels\dmol
  gcn          C:\Users\luobo\AppData\Roaming\jupyter\kernels\gcn
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> |
```

2. 在 VS Code 中使用 Jupyter Notebook

- (1) 定位至 gcn.ipynb 所在目录，右键打开菜单，选择“在终端中打开”。

```
(gcn) PS D:\progress\20240901-课题组\事务\知识工程\gcn> jupyter lab
[1] 2026-04-20 09:20:30.105 ServerApp jupyter_lsp | extension was successfully linked.
[2] 2026-04-20 09:20:30.110 ServerApp jupyter_server_terminals | extension was successfully linked.
[3] 2026-04-20 09:20:30.116 ServerApp jupyterlab | extension was successfully linked.
[4] 2026-04-20 09:20:30.120 ServerApp notebook | extension was successfully linked.
[5] 2026-04-20 09:20:30.624 ServerApp notebook_shim | extension was successfully linked.
[6] 2026-04-20 09:20:30.624 ServerApp panel.io.jupyter_server_extension | extension was successfully linked.
[7] 2026-04-20 09:20:30.681 ServerApp notebook_shim | extension was successfully loaded.
[8] 2026-04-20 09:20:30.683 ServerApp jupyter_lsp | extension was successfully loaded.
[9] 2026-04-20 09:20:30.684 ServerApp jupyter_server_terminals | extension was successfully loaded.
[10] 2026-04-20 09:20:30.691 LabApp JupyterLab extension loaded from D:\envs\scoop\apps\anaconda3\current\AppData\Local\Envs\gcn\share\jupyterlab
[11] 2026-04-20 09:20:30.691 LabApp JupyterLab application directory is D:\envs\scoop\apps\anaconda3\2025.12-2\AppData\Local\Envs\gcn\share\jupyterlab
```

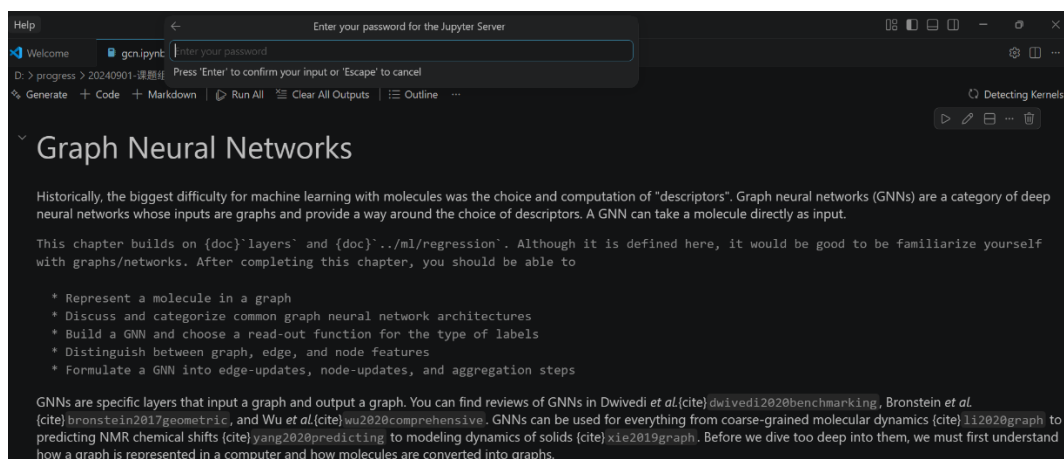
若右键菜单中无此选项，则可以打开一个新的命令行，进入所在目录。

```
PS C:\Users\luobo> d:
PS D:\> cd "D:\progress\20240901-课题组\事务\知识工程\gcn"
PS D:\progress\20240901-课题组\事务\知识工程\gcn> |
```

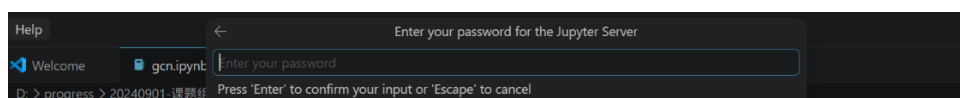
记录 token（地址中 token= 之后的部分，如本案例中 token 前 6 位为 634313）

```
To access the server, open this file in a browser:
  file:///C:/Users/luobo/AppData/Roaming/jupyter/runtime/jpserver-11336-open.html
Or copy and paste one of these URLs:
  http://localhost:8888/lab?token=63431311e3bce0aafe8d8e7fe01435b6bde9a0ca19a8f5a
  http://127.0.0.1:8888/lab?token=63431311e3bce0aafe8d8e7fe01435b6bde9a0ca19a8f5a
```

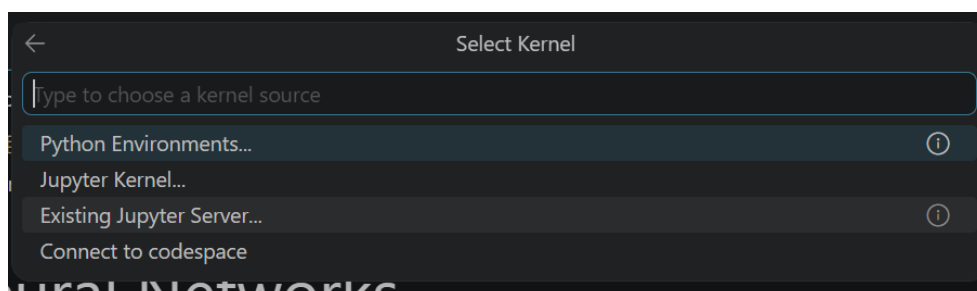
- (2) 在 vscode 中打开 gcn.ipynb 文件。



- (3) 新版 vscode 插件会自动检测 jupyter server 的信息，只需要在上方填写 token 即可。

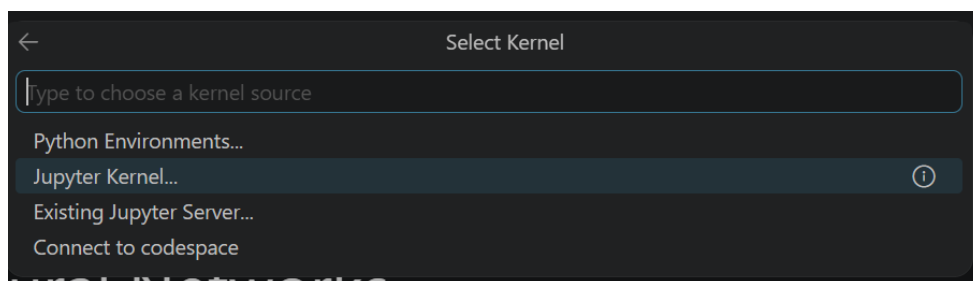


若并未自动弹出在右上角点击 select kernel -> Existing Jupyter Server，输入 server 地址。

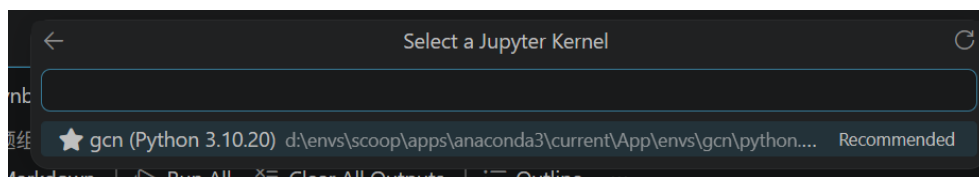


将 (1) 中记录的 token 作为 password 输入。

- (4) 选择 kernel。选择 select kernel -> jupyter kernels...



选择之前创建好的环境。



3. 实验原理与步骤

(1) 实验目的

本实验将利用 dmol 上的小型溶解度数据集，训练图卷积网络，完成化合物溶解度预测任务。

(2) 数据集介绍

dmol 上的溶解度数据集包含了 9982 条物质的 SMILES 结构、溶解度数据和一些其他的化学结构数据，可用于溶解度预测、空间群预测等任务。

(3) onehot 编码化合物

每一种化合物都可以视为是一系列元素原子相连而成（虽然在化学上考虑到离子键的存在这样并不严谨，但在 gcn 角度这样的简化是合理的），对应了图的结构。为了编码化合物的连接形态，需要编码元素类型和原子连接方式，对应图的顶点和边。对于元素类型，我们采用一种 one-hot 的方式，即只有对应元素的位置设置为 1，其他位置均为 0。对于连接，则采用邻接矩阵的形式，显然这是一个对称矩阵。

Node	C	H	O
1	0	1	0
2	0	1	0
3	0	1	0
4	1	0	0
5	0	0	1
6	0	1	0

$\mathbf{V}$ will be the combined feature vectors of these nodes. The adjacency matrix $\mathbf{E}$ will look like:

	1	2	3	4	5	6
1	0	0	0	1	0	0
2	0	0	0	1	0	0
3	0	0	0	1	0	0
4	1	1	1	0	1	0
5	0	0	0	1	0	1
6	0	0	0	0	1	0

(4) SMILES 结构表达式

我们知道，有机物不止有分子式，也同样有结构式，但结构式的书写并不适合于程序处理，因此需要将其序列化。而 SMILES 正是一种线性的化合物结构表达方式，它可以使用纯字符形式，表达化合物的结构形态，从而序列化结构信息。每一个 SMILES 表达式唯一的对应了一种化合物。

举一些常见的 SMILES 例子。CC(C)C(=O)O 这个 SMILES 表达式对应异丁酸，可以按照如下规则阅读。首先 SMILES 中氢被省略，圆括号对应于支链。因此这个 SMILES 表达式代表的化合物具有三个碳的主链，1 号和 2 号碳上分别有一个碳和羧基，即异丁酸。

(5) 有了如上的前置知识，就可以开始导入数据了。

```
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import pandas as pd
import rdkit, rdkit.Chem, rdkit.Chem.rdDepictor, rdkit.Chem.Draw
import networkx as nx
import dmol

soldata = pd.read_csv(
    "curated-solubility-dataset.csv"
)
# filter any whose smiles don't read in correctly (this changes depending on rdkit version, so we check here)
N = len(soldata)
soldata = soldata[soldata.SMILES.apply(lambda x: rdkit.Chem.MolFromSmiles(x) is not None)].reset_index(drop=True)
print(f"Dropped {N - len(soldata)} rows with invalid SMILES")

np.random.seed(0)
my_elements = {6: "C", 8: "O", 1: "H"}
```

由于数据集中存在极少量的非标准 SMILES 表达式，所以需要提前验证数据集做一次数据清洗。随后定义元素类型，用于设置 one-hot 编码。

可以通过一些简单的案例验证编码是否正确。

```
u = min(j.GetBeginAtomIdx(), j.GetEndAtomIdx())
v = max(j.GetBeginAtomIdx(), j.GetEndAtomIdx())
order = j.GetBondType()
if order in order_string:
    order = order_string[order]
else:
    raise Warning("Ignoring bond order" + order)
adj[u, v, order] = 1
adj[v, u, order] = 1
return nodes, adj

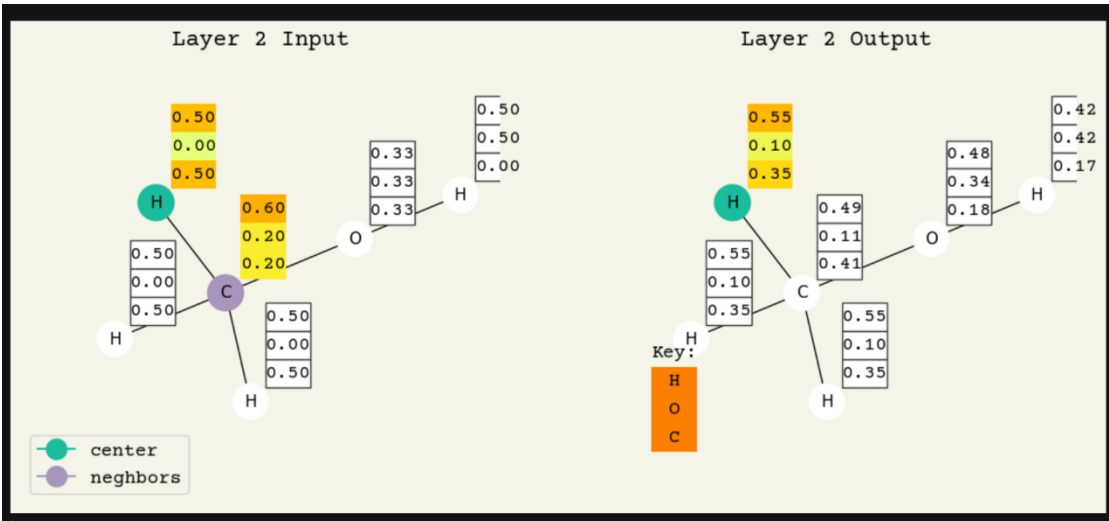
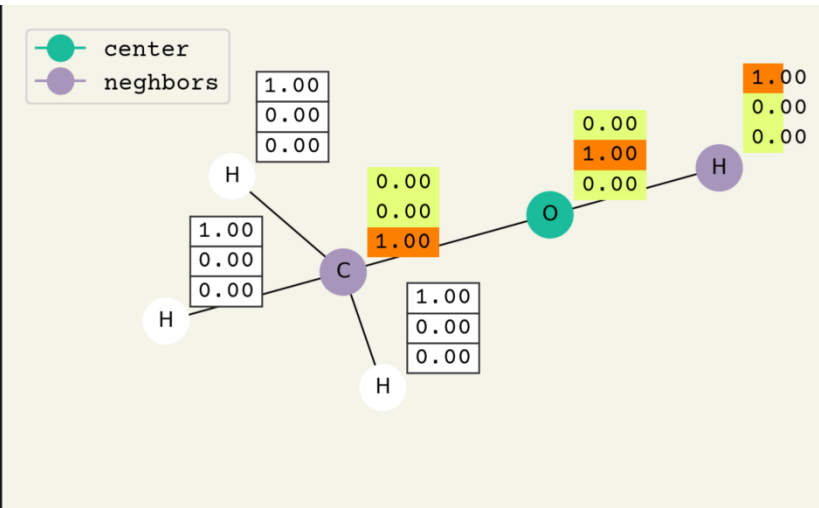
nodes, adj = smiles2graph("CO")
nodes
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 0., 1.]])
```

注意这里的 CO 出现了 6 个原子，是因为在 SMILES 里，氢都是被省略的。因此 CO 对应于甲酸 (CH3OH)，自然是 6 个原子。

(6) GNN 与 GCN

图神经网络（GNN）即为满足输出为图且能够保持图轮换性质不变（类似于线性变换保持线性一样，保持轮换意为调换顶点顺序，网络输出也只会调换顶点顺序）的网络。而图卷积网络（GCN），类似于 CNN 上的卷子操作，可以理解为每一层的更新都需要自身以及周边节点的特征。GCN 中最为经典的模型设计，即为 Kipf & Welling GCN。

关于 GCN 可以进一步做可视化



第二张图可以在运行后的同级目录的 gcn.gif 中看到动图。

(7) 训练 GCN

首先定义 GCN 层。

```

class GCNLayer(nn.Module):
    """Implementation of GCN as layer"""

    def __init__(self, in_features, out_features, activation=None):
        super(GCNLayer, self).__init__()
        self.w = nn.Parameter(torch.empty(in_features, out_features))
        nn.init.xavier_uniform_(self.w)
        self.activation = activation

    def forward(self, nodes, adj):
        degree = torch.sum(adj, dim=-1)
        new_nodes = torch.einsum("bi,bij,bjk,kl->bil", 1 / degree, adj, nodes, self.w)
        if self.activation is not None:
            new_nodes = self.activation(new_nodes)
        return new_nodes, adj

```

由于 GCN 只能是图输入对应图输出，因此需要一个将图转化为特征向量的层。

```

class GRLayer(nn.Module):
    """A GNN layer that computes average over all node features"""

    def __init__(self):
        super(GRLayer, self).__init__()

    def forward(self, nodes, adj):
        reduction = torch.mean(nodes, dim=1)
        return reduction

```

定义主干模型。

```

class SolubilityModel(nn.Module):
    def __init__(self, node_features=100):
        super(SolubilityModel, self).__init__()
        self.gcn1 = GCNLayer(node_features, node_features, activation=F.relu)
        self.gcn2 = GCNLayer(node_features, node_features, activation=F.relu)
        self.gcn3 = GCNLayer(node_features, node_features, activation=F.relu)
        self.gcn4 = GCNLayer(node_features, node_features, activation=F.relu)
        self.gr = GRLayer()
        self.fc1 = nn.Linear(node_features, 16)
        self.fc2 = nn.Linear(16, 1)

    def forward(self, nodes, adj):
        x, adj = self.gcn1(nodes, adj)
        x, adj = self.gcn2(x, adj)
        x, adj = self.gcn3(x, adj)
        x, adj = self.gcn4(x, adj)
        x = self.gr(x, adj)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SolubilityModel()

```

加载数据集。


```

from torch.utils.data import Dataset, DataLoader

class SolubilityDataset(Dataset):
    def __init__(self, data, start_idx=0, end_idx=None):
        self.data = data
        self.start_idx = start_idx
        self.end_idx = end_idx if end_idx is not None else len(data)

    def __len__(self):
        return self.end_idx - self.start_idx

    def __getitem__(self, idx):
        idx = idx + self.start_idx
        nodes, adj = gen_smiles2graph(self.data.SMILES[idx])
        sol = self.data.Solubility[idx]
        # this time we do not batch - it will be done by dataloader
        nodes_t = torch.from_numpy(nodes).float()
        adj_t = torch.from_numpy(adj).float()
        return (nodes_t, adj_t), torch.tensor(sol, dtype=torch.float32)

```

定义训练循环。

```

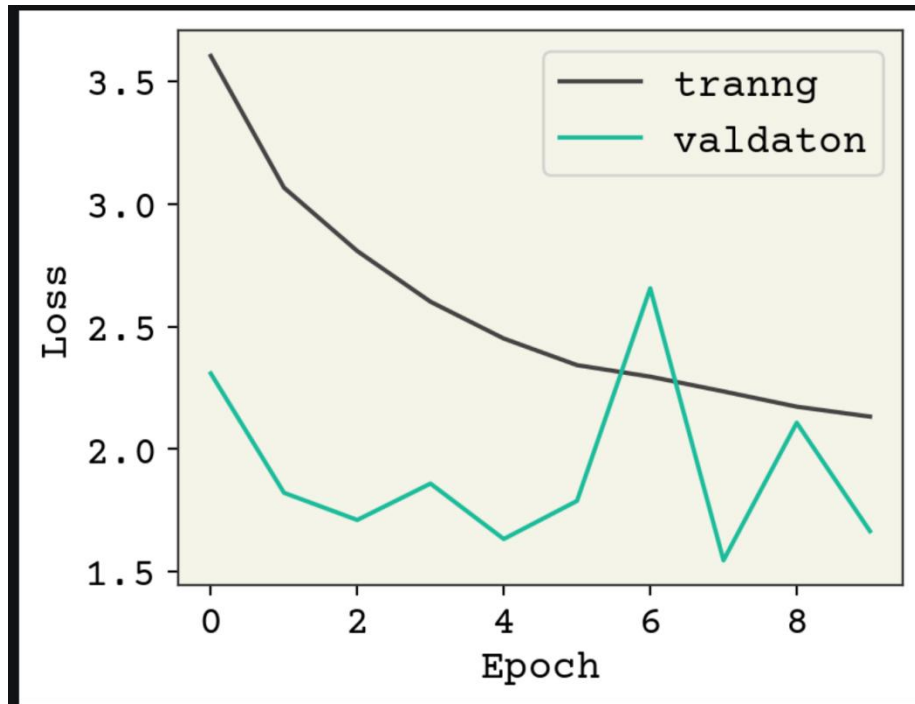
for epoch in range(epochs):
    model.train()
    train_loss = 0
    for (nodes, adj), y in train_loader:
        optimizer.zero_grad()
        y_pred = model(nodes, adj)
        loss = criterion(y_pred.squeeze(), y)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()

    model.eval()
    val_loss = 0
    with torch.no_grad():
        for (nodes, adj), y in val_loader:
            y_pred = model(nodes, adj)
            loss = criterion(y_pred.squeeze(), y)
            val_loss += loss.item()

    train_loss /= len(train_loader)
    val_loss /= len(val_loader)
    history["loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    print(f"Epoch {epoch+1}/{epochs}, Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}")

```

(8) 查看训练情况和测试结果

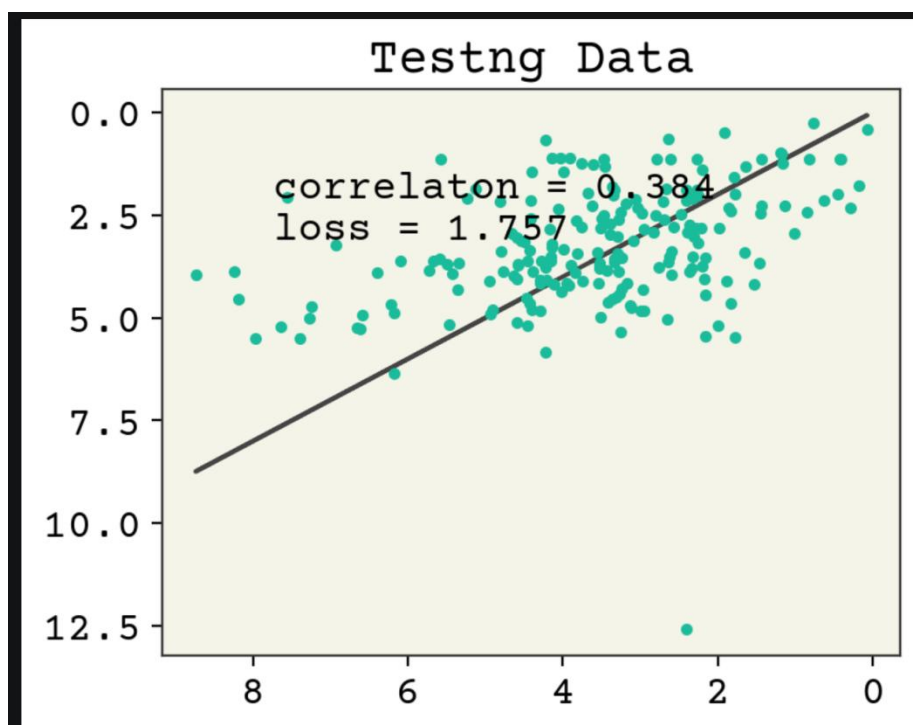


```

model.eval()
yhat = []
test_y = []
test_loader = DataLoader(test_data, batch_size=1, shuffle=False)
with torch.no_grad():
    for (nodes, adj), y in test_loader:
        pred = model(nodes, adj)
        yhat.append(pred.squeeze().item())
        test_y.append(y.item())

plt.figure()
plt.plot(test_y, test_y, "-")
plt.plot(test_y, yhat, ".")
plt.text(
    min(test_y) + 1,
    max(test_y) - 2,
    f"correlation = {np.corrcoef(test_y, yhat)[0,1]:.3f}",
)
plt.text(
    min(test_y) + 1,
    max(test_y) - 3,
    f"loss = {np.sqrt(np.mean((np.array(test_y) - np.array(yhat))**2)):.3f}",
)
plt.title("Testing Data")
plt.show()

```



可以看出过拟合还是比较严重的，主要是因为模型为了能够支持任意长度的化合物，其节点数量是动态的。对较长数据进行训练时，就会引入大量的参数，使得模型的表达能力远远超过数据集特征的表达能力，进而引发了过拟合。

4. 执行 notebook 代码

【实验任务】完成 notebook 文件 gcn.ipynb 中的内容：

将 notebook 每个代码段执行成功的输出截图粘贴在下面

（说明：代码段【19】和【37】运行时间稍长，同学们耐心等待，尤其【37】，大概运行 20min 左右。）

【代码段 1 执行截图】：

```
import pandas as pd
import rdkit, rdkit.Chem, rdkit.Chem.rdDepictor, rdkit.Chem.Draw
import networkx as nx
import dmol

✓ 2.0s

soldata = pd.read_csv(
```

【代码段 2 执行截图】：

```
[2] ✓ 0.5s  
... Dropped 2 rows with invalid SMILES
```

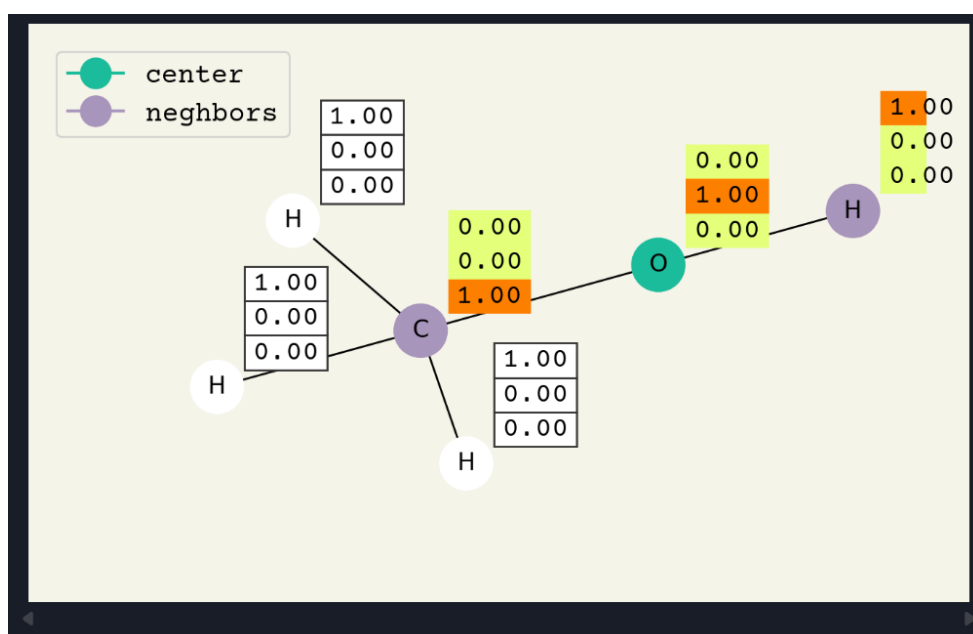
【代码段 3 执行截图】

```
adj[v, 0, order] = 1  
return nodes, adj  
✓ 0.0s × hide-cell + Tag
```

【代码段 4 执行截图】

```
[4] ✓ 0.0s  
... array([[1., 0., 0.],  
          [0., 1., 0.],  
          [0., 0., 1.],  
          [0., 0., 1.],  
          [0., 0., 1.],  
          [0., 0., 1.]])
```

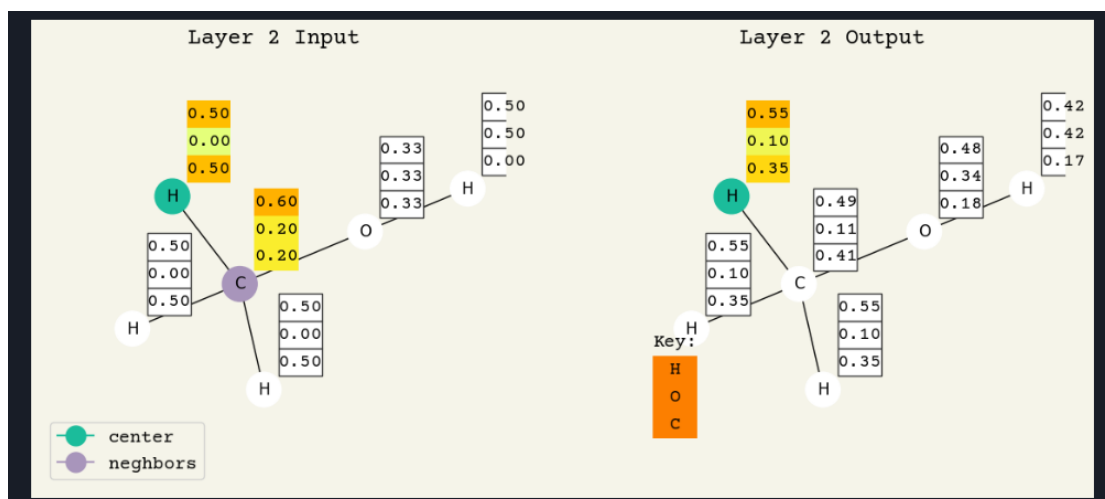
【代码段 5 执行截图】



【代码段 6 执行截图】

```
[6] ✓ 9.8s ✕ remove-cell + Tag

... [0] 0 5 0
[False, False, False, False, False, True] 5
MoviePy - Building file gcn.gif with imageio
```



【代码段 7 执行截图】

```
[7] ✓ 0.0s

... array([[1., 1., 1., 1., 1., 0.],
          [1., 1., 0., 0., 0., 1.],
          [1., 0., 1., 0., 0., 0.],
          [1., 0., 0., 1., 0., 0.],
          [1., 0., 0., 0., 1., 0.],
          [0., 1., 0., 0., 0., 1.]])
```

【代码段 8 执行截图】

```
1 ✓ 0.0s

array([5., 3., 2., 2., 2., 2.])
```

【代码段 9 执行截图】

```
[9]  ✓ 0.0s

...  [1.  0.  0.]
      [0.2 0.2 0.6]
```

【代码段 10 执行截图】

```
✓ 0.0s

A lot of the code above is PyTorch specific and getting the variables to the right place. There are
```

【代码段 11 执行截图】

```
(tensor([[[0.1338, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000],
          [0.0000, 0.0000, 0.0000],
          [0.0513, 0.0000, 0.1117]]], grad_fn=<ReluBackward0>),
 tensor([[[1., 1., 1., 1., 1., 0.],
          [1., 1., 0., 0., 0., 1.],
          [1., 0., 1., 0., 0., 0.],
          [1., 0., 0., 1., 0., 0.],
          [1., 0., 0., 0., 1., 0.],
          [0., 1., 0., 0., 0., 1.]]]]))
```

【代码段 12 执行截图】

```
[12]  ✓ 0.0s

...  tensor([[[0.0000, 0.0197, 0.0000],
          [0.0000, 0.0154, 0.0000],
          [0.0000, 0.0493, 0.0000],
          [0.0000, 0.0493, 0.0000],
          [0.0000, 0.0493, 0.0000],
          [0.0174, 0.0000, 0.0000]]], grad_fn=<ReluBackward0>)
```

【代码段 13 执行截图】

```
return reduction
```

[13] ✓ 0.0s Pyt

【代码段 14 执行截图】

[14] ✓ 0.0s

【代码段 15 执行截图】

[15] ✓ 0.0s × μ!qq6u-c6ll + J9d

【代码段 16 执行截图】

```
... f6u20L([[ -θ- Tθθ8]])' 8L9q~tu=<AqQwWBgCjKw9Lqθ>)
```

[16] ✓ 0.0s

【代码段 17 执行截图】

[17] ✓ 0.0s

【代码段 18 执行截图】

```
train_data = sobolnityDatabase
```

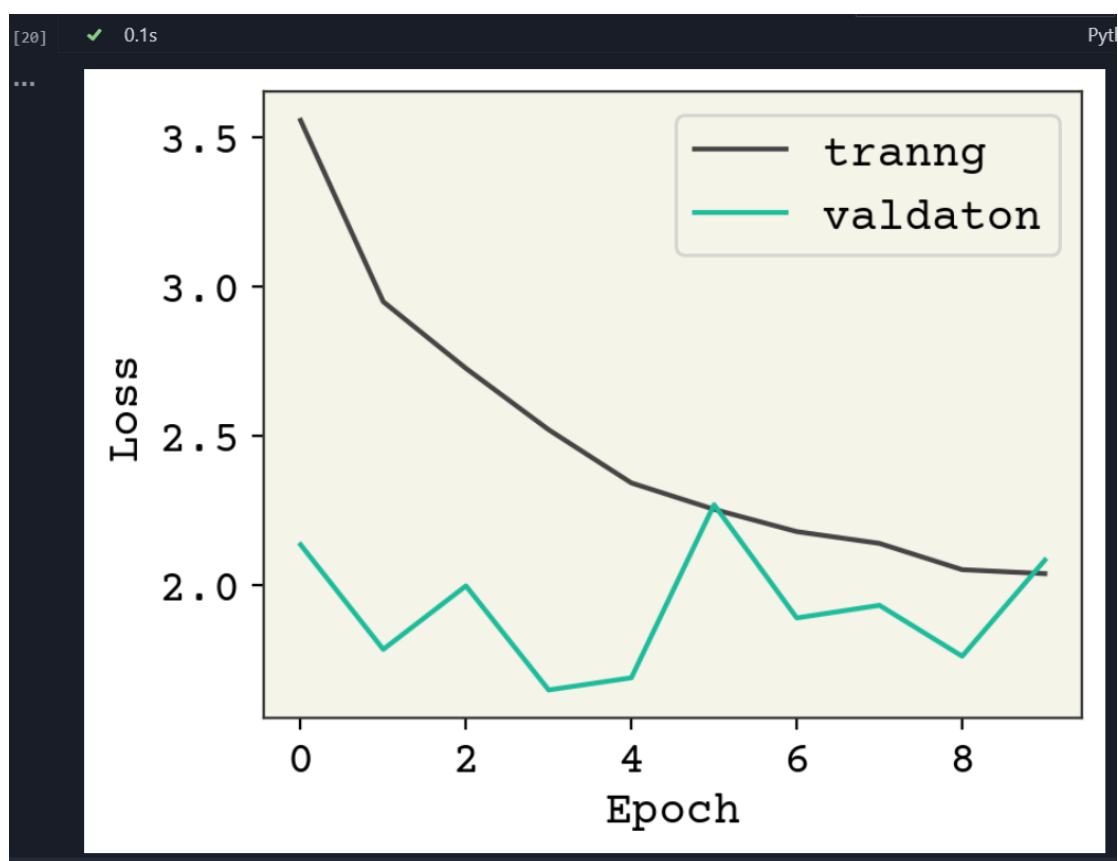
[18] ✓ 0.0s

【代码段 19 执行截图】

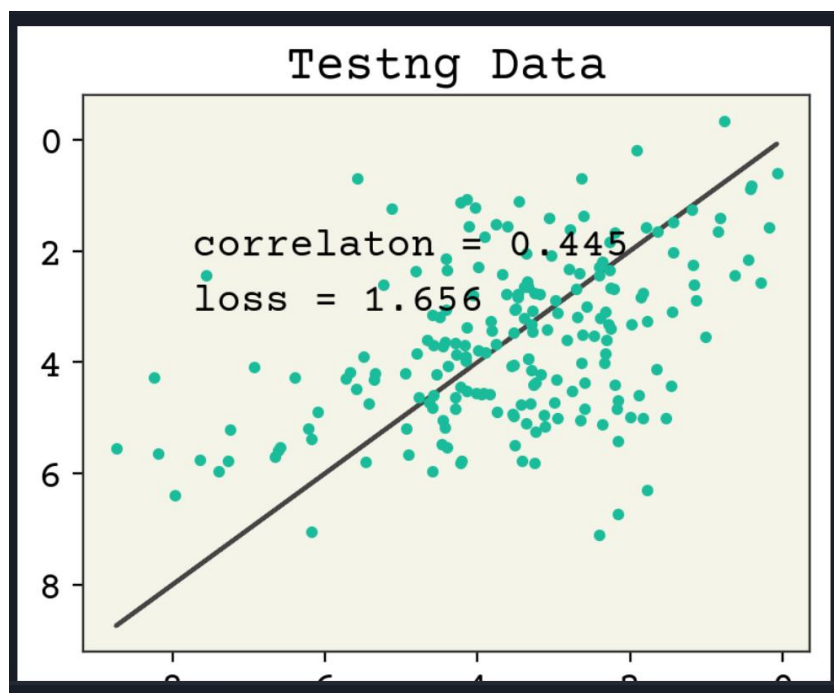
[19] ✓ 3m 45.8s × remove-output + Tag

```
Epoch 1/10, Train Loss: 3.5566, Val Loss: 2.1360
Epoch 2/10, Train Loss: 2.9494, Val Loss: 1.7855
Epoch 3/10, Train Loss: 2.7262, Val Loss: 1.9980
Epoch 4/10, Train Loss: 2.5208, Val Loss: 1.6495
Epoch 5/10, Train Loss: 2.3426, Val Loss: 1.6904
Epoch 6/10, Train Loss: 2.2542, Val Loss: 2.2698
Epoch 7/10, Train Loss: 2.1797, Val Loss: 1.8907
Epoch 8/10, Train Loss: 2.1402, Val Loss: 1.9330
Epoch 9/10, Train Loss: 2.0523, Val Loss: 1.7625
Epoch 10/10, Train Loss: 2.0390, Val Loss: 2.0843
```

【代码段 20 执行截图】



【代码段 21 执行截图】

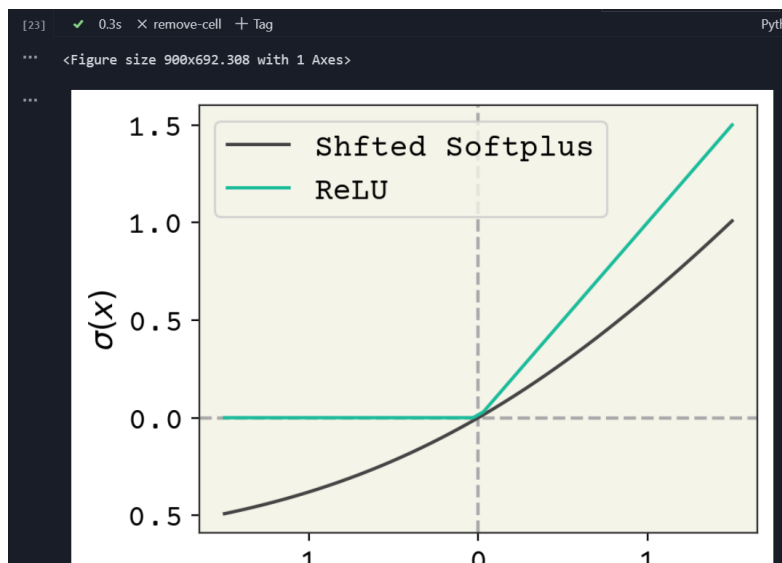


【代码段 22 执行截图】

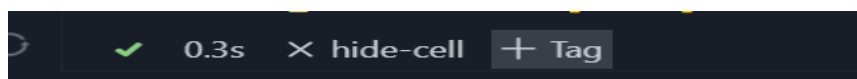
```
[22] ✓ 0.0s

... input 2
      output [0.02 0.78 0.  0.  0.  ]
```

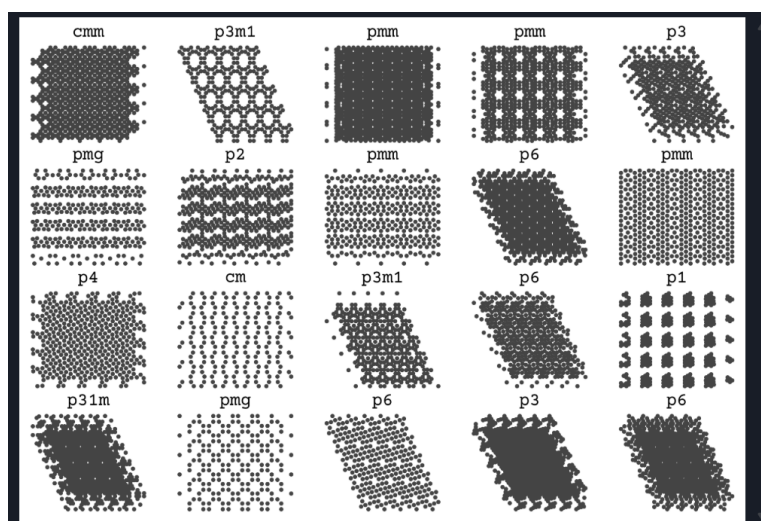
【代码段 23 执行截图】



【代码段 24 执行截图】



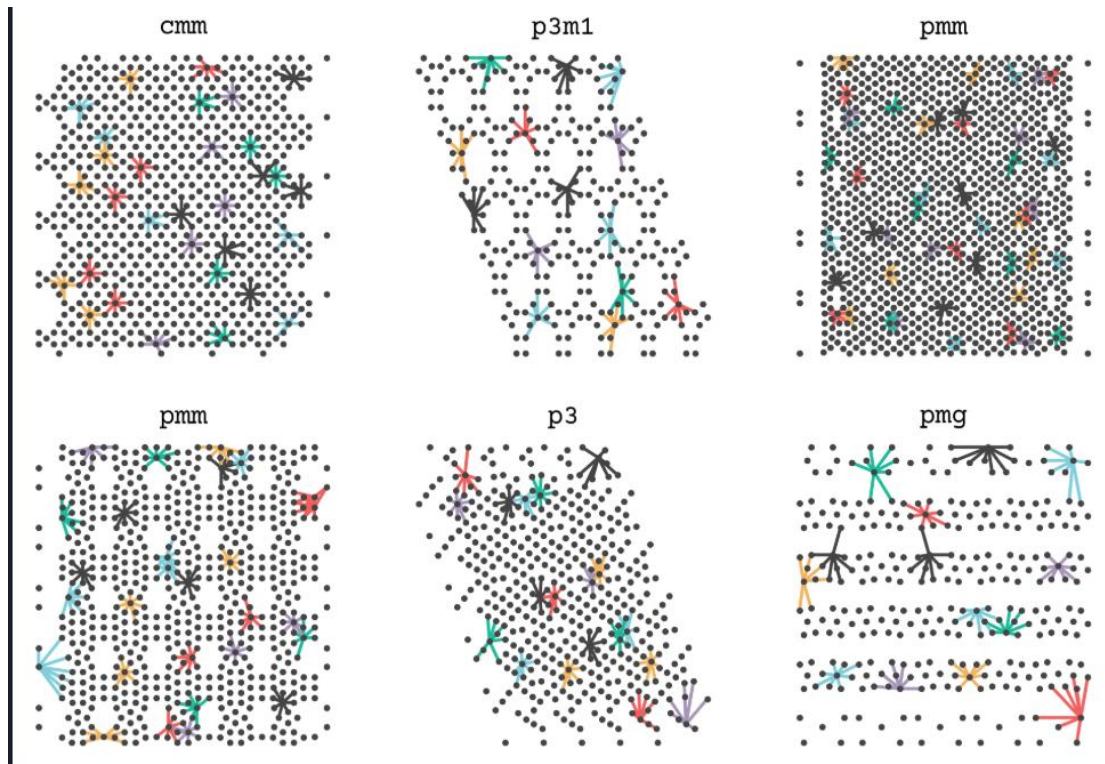
【代码段 25 执行截图】



【代码段 26 执行截图】

```
[26] ✓ 0.0s
```

【代码段 27 执行截图】



【代码段 28 执行截图】

```
[28] ✓ 3.7s
```

【代码段 29 执行截图】

```
[29] ✓ 0.0s Python
... first node: 1
first node, first edge features: [3.3756149e-01 3.7093274e-02 3.3091463e-10 2.3967226e-25 0.0000000e+00 0.0000000e+00 0.0000000e+00]
first node, all neighbors [ 28 668 7 25 617 4 2 627 658 661 670 24 642 683 3 0]
label 15
```

【代码段 30 执行截图】

```
[30] ✓ 0.0s Python
```

【代码段 31 执行截图】

```
[31] ✓ 0.0s Python
```

【代码段 32 执行截图】

```
[32] ✓ 0.0s
```

【代码段 33 执行截图】

```
[33] ✓ 0.0s
... [-0.03774558  0.21041043  0.07815225 -0.01460974  0.14682174 -0.25519732
      0.02660044  0.06863152  0.16544119 -0.07488122  0.09850977 -0.36537114
      0.03327708 -0.14085932 -0.18678974  0.19094324  0.21789159]
```

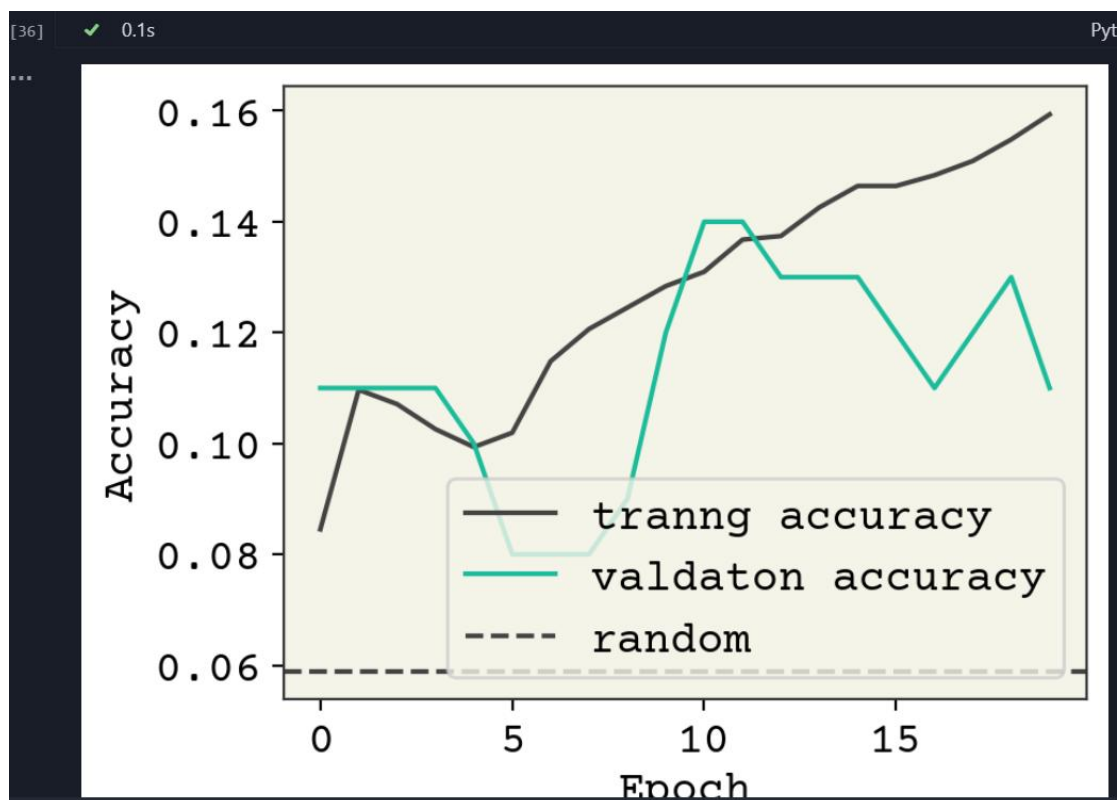
【代码段 34 执行截图】

```
print("predicted class", F.softmax(yhat, dim=0).detach().numpy())
[34] ✓ 0.0s
... predicted class [0.05538704 0.07098735 0.06219307 0.05668341 0.06661388 0.04456256
                    0.05906814 0.06160375 0.06786581 0.05336793 0.06347214 0.03991372
                    0.05946384 0.04996046 0.04771766 0.06961879 0.07152041]
```

【代码段 35 执行截图】

```
[35] ✓ 10m 38.5s ✕ remove-output + Tag
... Epoch 1/20, Train Acc: 0.0845, Val Acc: 0.1100
Epoch 2/20, Train Acc: 0.1097, Val Acc: 0.1100
Epoch 3/20, Train Acc: 0.1071, Val Acc: 0.1100
Epoch 4/20, Train Acc: 0.1026, Val Acc: 0.1100
Epoch 5/20, Train Acc: 0.0994, Val Acc: 0.1000
Epoch 6/20, Train Acc: 0.1019, Val Acc: 0.0800
Epoch 7/20, Train Acc: 0.1148, Val Acc: 0.0800
Epoch 8/20, Train Acc: 0.1206, Val Acc: 0.0800
Epoch 9/20, Train Acc: 0.1245, Val Acc: 0.0900
Epoch 10/20, Train Acc: 0.1284, Val Acc: 0.1200
Epoch 11/20, Train Acc: 0.1310, Val Acc: 0.1400
Epoch 12/20, Train Acc: 0.1368, Val Acc: 0.1400
Epoch 13/20, Train Acc: 0.1374, Val Acc: 0.1300
Epoch 14/20, Train Acc: 0.1426, Val Acc: 0.1300
Epoch 15/20, Train Acc: 0.1465, Val Acc: 0.1300
Epoch 16/20, Train Acc: 0.1465, Val Acc: 0.1200
Epoch 17/20, Train Acc: 0.1484, Val Acc: 0.1100
Epoch 18/20, Train Acc: 0.1510, Val Acc: 0.1200
Epoch 19/20, Train Acc: 0.1548, Val Acc: 0.1300
Epoch 20/20, Train Acc: 0.1594, Val Acc: 0.1100
```

【代码段 36 执行截图】



【代码段 37 执行截图】

